Lab 3: Instruction Set

- **Link**
  - document
  - video
  - datasheet

- **Basic** (50%)

  - **Description**

    Implement an adjacent-bit swap operation. An 8-bit unsigned integer is stored at memory address [0x020]. Swap every pair of adjacent bits and store the resulting 8-bit value at memory address [0x021]. Specifically, the required bit transformation is as follows:

    Before:

    bit7 bit6 bit5 bit4 bit3 bit2 bit1 bit0
    a b c d e f g h

    After:

    bit7 bit6 bit5 bit4 bit3 bit2 bit1 bit0

    b a d c f e h g

    In other words, bits 7 and 6, bits 5 and 4, bits 3 and 2, and bits 1 and 0 must be swapped with each other.

  - **Example**

Address	Testcase 1	Testcase 2
[0x020]	0xB2	0x6D
[0x021]	0x71	0x9E

Test Case 1 :

Input = $(0xB2)_{16} = (10110010)_2$

Output = $(0x71)_{16} = (01110001)_2$

Test Case 2 :

Input = $(0x6D)_{16} = (01101101)_2$

Output = $(0x9E)_{16} = (10011110)_2$

■ **Standard of grading**

1. The input value must be stored at memory address [0x020], and the final result must be stored at memory address [0x021].
2. Your implementation must use at least one appropriate bit-manipulation instruction, such as AND, OR, XOR, or ROTATE instruction.
3. Do not use branch-related instructions, including: GOTO, BRA, CPFSEQ, CPFSGT, CPFSLT, and TSTFSZ
4. Points will be deducted if any of these requirements are violated.

- **Advance** (30%)

■ **Description**

Implement a 16-bit signed absolute-value operation.

A 16-bit signed integer in two's complement representation form is stored in memory locations [0x000] and [0x001], where 0x000 contains the high-order byte and 0x001 contains the low-order byte. Compute the absolute magnitude of the input value and store the resulting 16-bit value at memory addresses [0x010] and[0x011], with the high-order byte stored at 0x010 and the low-order byte stored at 0x011.

■ **Example**

Address	Test Case 1	Test Case 2
[0x000]	0x00	0xFF
[0x001]	0x64	0x9C
[0x010]	0x00	0x00
[0x011]	0x64	0x64

Testcase 1 :

Input = 0x0064 = 100

Output = 0x0064 = 100

Testcase 2 :

Input = 0xFF9C = -100

Output = 0x0064 = 100

■ **Standard of grading**

1. Store the input in memory locations [0x000] and [0x001], and store the result in [0x010] and [0x011].
2. The program must correctly compute the absolute magnitude of the 16-bit signed input without modifying the original value.
3. The instructions **NEGF, SUBWF, and SUBWFB** are not allowed.
4. Do not modify the original input.
5. When converting a negative value using two's complement, your implementation must correctly propagate the carry generated when incrementing the low-order byte into the high-order byte when necessary.

■ **Hint**

You may first determine whether the 16-bit input is negative. If the value is negative , its absolute magnitude can be obtained using the **two's complement relationship**:

Two's complement = One's complement + 1

- **Hard** (20%)

■ **Description**

SUBLEQ is a Turing-complete instruction set consisting of a single instruction:

```
subleq a, b, c
```

This instruction means: set *b to *b - *a, and if *b is less than or equal to 0, execution branches to address c; otherwise, execution continues the next instruction. Here,*a and *b denote the values stored at memory addresses a and b, respectively.

In this problem, you are required to implement a SUBLEQ interpreter according to the precise execution semantics described below. Given an initial memory state, your program must execute SUBLEQ instructions until the termination condition is reached, leaving the simulated memory in its final state.

Each SUBLEQ instruction occupies exactly three consecutive bytes. From the lowest to the highest memory address, these three bytes represent a, b, and c, respectively. Each operand occupies one byte and therefore specifies an address in the range 0x00 through 0xFF.

PC represents the address of the next SUBLEQ instruction to be executed. It is an abstract program counter value for the simulated SUBLEQ machine, and you may implement it using any appropriate mechanism, provided its behavior follows the specification below.

The initial value of the PC is 0x00. Before each instruction is executed, the termination condition is checked. If the PC is greater than or equal to 0xFE, execution terminates. Otherwise, the three bytes beginning at the address specified by PC are fetched and interpreted as a, b, and c.

To execute an instruction, interpret the values stored at memory addresses a and b as 8-bit signed two's-complement integers. Compute *b - *a and store the lower eight bits of the result back at memory address b. If arithmetic overflow occurs, the result naturally wraps modulo 282^8, as in normal 8-bit two's-complement arithmetic.

After the subtraction, examine the resulting 8-bit value stored at address b. If the result is zero or its most significant bit is 1, set PC to c; otherwise, set PC to PC + 3. Note that PC itself is a non-negative integer that does not overflow: if the current PC is 0xFD and execution proceeds sequentially, the next value of PC must be 0x100, after which the termination condition is satisfied.

The instruction memory and data memory of the simulated SUBLEQ machine share the same address space, consisting of file registers 0x00 through 0xFF. Therefore, self-modifying code is permitted. However, you may assume that an instruction will never modify any of the three bytes belonging to the instruction currently being executed.

The input consists of the initial values of memory addresses 0x00 through 0xFF. Any memory address whose initial value is not explicitly specified should be assumed to contain 0x00.

After execution terminates, memory addresses 0x00 through 0xFF must contain exactly the final state produced by the simulated SUBLEQ program.

■ Example

The code template and the explanation of the test cases are shown on the HackMD: https://hackmd.io/@rota1001/mcu2026-lab3-hard-subleq
You are highly encouraged to follow the provided code template. During the demo, we will use the same format to give you the input.

Test Case 1:

Input:

Address	[0x00]	[0x01]	[0x02]	[0x03]	[0x04]	[0x05]
Value	0x04	0x05	0x03	0x01	0x02	0x00

Result:

Address	[0x02]	[0x05]
Value	0xFE	0xFE

Test Case 2:

Input:

Address	[0x00]	[0x01]	[0x02]	[0x03]	[0x04]	[0x05]
Value	0x09	0x0C	0x03	0x0A	0x0B	0xFE

Address	[0x06]	[0x07]	[0x08]	[0x09]	[0x0A]	[0x0B]
Value	0x0D	0x0D	0x00	0xFF	0x01	0x05

Result:

Address	[0x0B]	[0x0C]	[0x0D]
Value	0x00	0x05	0x00

■ Hint

Remember that memory addresses 0x00 through 0xFF belong to the simulated SUBLEQ machine and must contain the correct final state after execution terminates. Therefore, any additional state required by your interpreter, such as PC or temporary variables, should be stored outside this memory region or maintained in a way that does not corrupt the simulated memory.

■ Standard of grading

1. The final contents of memory addresses **[0x00] through [0xFF]** must match the expected state after the SUBLEQ program terminates. During the demonstration, the teaching assistants will verify selected memory locations.
2. You may use any implementation strategy and any appropriate PIC18F instructions, provided that the resulting behavior strictly follows the SUBLEQ execution semantics defined above.